

electron/git-publish.cjs

Kind: added · Baseline: 8e0dc031dcfc · Target: NovaDiff · Generated: 2026-05-19T21:12:13.783Z

Overview

A new module `electron/git-publish.cjs` is added. It exports three functions:

`buildFullCommitMessage`, `previewPublish`, and `executePublish`.

Key changes

- **Imports (lines 3-5):** `const path = require("node:path"); const { getRepoStatus, stageAll, commit, push, isGitRepo } = require("./git-service.cjs"); const { createPullRequest, getAuthStatus, slugFromRepoRoot } = require("./github-service.cjs");`

````- **buildFullCommitMessage (lines 7-13)**: trims subject and body; returns subject alone if body is empty, otherwise inserts a blank line between them.`

- **previewPublish (lines 19-42):** resolves `repoRoot`, verifies a Git repo, gathers status, GitHub auth, and slug, then returns an object containing `repoRoot`, `branch`, `upstream`, `ahead`, `behind`, `dirtyFiles`, `remotes`, `github`, `githubSlug`, and `canPush`.
- **executePublish (lines 48-101):** resolves `repoRoot`, `subject`, and `body`; validates the subject; stages all files, commits with the built message, optionally pushes to a remote, and optionally creates a PR via `gh`. It returns an object with `ok`, `commitHash`, `branch`, `pushed`, `pushRemote`, and `prUrl`.
- **Exports (lines 104-108):** `module.exports = { previewPublish, executePublish, buildFullCommitMessage };`

### Impact

- Provides a clear commit-message builder and a publish workflow that respects `.gitignore` through `stageAll`.
- `previewPublish` offers a snapshot of the repository state, useful for debugging and CI checks.
- `executePublish` requires the GitHub CLI to be authenticated; it throws an error if `gh` is not logged in.
- The default remote is `"origin"` unless overridden by `opts.remote`.

### Risks & follow-ups

- **Authentication:** `executePublish` throws if `gh` is not logged in; CI environments must run `gh auth login`.
- **Remote default:** The hard-coded `"origin"` may not match all repository conventions; verify usage.
- **Error handling:** Only basic checks are performed; consider wrapping Git commands in `try/catch` to surface underlying errors.

- **Side effects:** The module stages all files and commits unconditionally; tests should confirm that this behavior aligns with user expectations.